Exp-5 Dynamic Programming using Python

Problem 1: Household Appliances Knapsack

Aim

To determine the maximum value obtainable using Dynamic Programming for a knapsack of limited capacity.

Code:

```
[9]
def knapsack(weights, values, capacity):
    precision = 0
    for val in weights + [capacity]:
        s = str(val)
        if '.' in s:
            precision = max(precision, len(s.split('.')[1]))

    multiplier = 10 ** precision

    int_weights = [int(round(w * multiplier)) for w in weights]
    int_capacity = int(round(capacity * multiplier))

    n = len(values)
    dp = [[0] * (int_capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(int_capacity + 1):
            if int_weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                values[i-1] + dp[i-1][w - int_weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][int_capacity]

weights = [1.2, 1.5, 3.0, 2.0, 2.5]
values = [5, 4, 10, 6, 8]
capacity = 6
print(knapsack(weights, values, capacity))

... 19
```

Output:

19

Problem 2: Laptop Accessories Selection

Aim

To maximize the total value of selected accessories using Dynamic Programming.

Code:

```
[10]
0s
def knapsack(weights, values, capacity):
    precision = 0
    for val in weights + [capacity]:
        s = str(val)
        if '.' in s:
            precision = max(precision, len(s.split('.')[1]))

    multiplier = 10 ** precision

    int_weights = [int(round(w * multiplier)) for w in weights]
    int_capacity = int(round(capacity * multiplier))

    n = len(values)
    dp = [[0] * (int_capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(int_capacity + 1):
            if int_weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                values[i-1] + dp[i-1][w - int_weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][int_capacity]
weights = [1.8, 0.3, 0.7, 0.6, 1.5]
values = [7, 2, 4, 3, 6]
capacity = 6
print(knapsack(weights, values, capacity))

22
```

Output:

22

Problem 3: Medical Kit Packing

Aim:

To select medical supplies that give maximum benefit within a fixed weight limit.

Code:

```
[12]
0s
def knapsack(weights, values, capacity):
    precision = 0
    for val in weights + [capacity]:
        s = str(val)
        if '.' in s:
            precision = max(precision, len(s.split('.')[1]))

    multiplier = 10 ** precision

    int_weights = [int(round(w * multiplier)) for w in weights]
    int_capacity = int(round(capacity * multiplier))

    n = len(values)
    dp = [[0] * (int_capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(int_capacity + 1):
            if int_weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                values[i-1] + dp[i-1][w - int_weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][int_capacity]
weights = [1.0, 1.5, 0.5, 2.0, 0.3]
values = [5, 7, 3, 8, 2]
capacity = 4

print(knapsack(weights, values, capacity))

...  18
```

Output:

18

Problem 4: Tourist Backpack Problem

Aim:

To maximize utility of tourist items using DP.

Code:

```
def knapsack(weights, values, capacity):
    precision = 0
    for val in weights + [capacity]:
        s = str(val)
        if '.' in s:
            precision = max(precision, len(s.split('.')[1]))

    multiplier = 10 ** precision

    int_weights = [int(round(w * multiplier)) for w in weights]
    int_capacity = int(round(capacity * multiplier))

    n = len(values)
    dp = [[0] * (int_capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(int_capacity + 1):
            if int_weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                values[i-1] + dp[i-1][w - int_weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][int_capacity]
weights = [1.2, 2.5, 2.0, 1.0, 0.4]
values = [9, 6, 5, 3, 2]
capacity = 7

print(knapsack(weights, values, capacity))
```

```
23
```

Output:

23

Problem 5: Food Supply Knapsack

Aim:

To choose food items that maximize value within weight constraints.

Code:

```
[14]
def knapsack(weights, values, capacity):
    precision = 0
    for val in weights + [capacity]:
        s = str(val)
        if '.' in s:
            precision = max(precision, len(s.split('.')[1]))

    multiplier = 10 ** precision

    int_weights = [int(round(w * multiplier)) for w in weights]
    int_capacity = int(round(capacity * multiplier))

    n = len(values)
    dp = [[0] * (int_capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(int_capacity + 1):
            if int_weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                values[i-1] + dp[i-1][w - int_weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][int_capacity]
weights = [4.0, 3.5, 2.0, 1.5, 1.0]
values = [8, 7, 6, 5, 4]
capacity = 10

print(knapsack(weights, values, capacity))

...  24
```

Output:

24

Problem 6: Electronic Gadgets Selection

Aim:

To maximize the total value of gadgets in a limited-capacity bag.

Code:

```
[15]
0s
def knapsack(weights, values, capacity):
    precision = 0
    for val in weights + [capacity]:
        s = str(val)
        if '.' in s:
            precision = max(precision, len(s.split('.')[1]))

    multiplier = 10 ** precision

    int_weights = [int(round(w * multiplier)) for w in weights]
    int_capacity = int(round(capacity * multiplier))

    n = len(values)
    dp = [[0] * (int_capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(int_capacity + 1):
            if int_weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                values[i-1] + dp[i-1][w - int_weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][int_capacity]
weights = [1.0, 0.4, 1.2, 2.0, 0.8]
values = [7, 4, 6, 9, 3]
capacity = 6


print(knapsack(weights, values, capacity))

...  29
```

Output:

29

Problem 7: Student Study Kit

Aim:

To select study materials that give maximum utility using DP.

Code:

```
[16]
✓ 0s
def knapsack(weights, values, capacity):
    precision = 0
    for val in weights + [capacity]:
        s = str(val)
        if '.' in s:
            precision = max(precision, len(s.split('.')[1]))

    multiplier = 10 ** precision

    int_weights = [int(round(w * multiplier)) for w in weights]
    int_capacity = int(round(capacity * multiplier))

    n = len(values)
    dp = [[0] * (int_capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(int_capacity + 1):
            if int_weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                values[i-1] + dp[i-1][w - int_weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][int_capacity]
weights = [2.5, 1.0, 0.5, 0.6, 0.8]
values = [8,4,3,2,3]
capacity = 5


print(knapsack(weights, values, capacity))
... 18
```

Output:

18